

基于 Gradio 的垃圾分类识别系统

实验文档（四）推理功能实现

实现步骤总览

步骤	内容	验证方式
0.4	环境自检	有可用权重、 <code>records</code> 表存在
1.1	识别图片保存与表格工具	目录可写
1.2	<code>_recognize</code> 识别回调	函数可 import
1.3	记录查询、详情、导出	<code>python -c</code> 查记录数
1.4	<code>_build_records_panel_state</code>	—
2.1	「图像识别」页签 UI	页面可见控件
2.2	「识别记录」页签 UI	页面可见表格
2.3	<code>build_app</code> 注册与登录后加载记录	登录后记录表有数据
3.1	功能测试 IN1-IN8	测试表
3.2	常见问题排查	—
选做	接入真实 PyTorch 推理	见附录 C

学习路径

章节	目标
第 0 章	理解「识别 → 入库 → 记录页回放」闭环
第 1 章	后端：保存图片、识别回调、记录 CRUD
第 2 章	前端：两个页签的组件与事件
第 3 章	联调测试；选做接入真实模型

第 0 章 功能架构

0.1 业务功能一览

说明：推理模块由「图像识别」页签（发起识别）与「识别记录」页签（历史回放）组成。识别时使用模型管理模块提供的权重列表；识别结果写入 `records` 表，图片快照存到 `records/images/`。

功能	界面位置	后端函数
选择权重	图像识别 → 「权重文件」	<code>_refresh_weights / _get_current_weight</code>
上传并识别	图像识别 → 「开始识别」	<code>_recognize</code>
保存识别快照	<code>_recognize</code> 内部	<code>_save_record_image</code>
写入识别记录	<code>_recognize</code> 内部	<code>db.create_record</code>
分页查询记录	识别记录 → 「查询」	<code>_records_query</code>
查看单条详情	识别记录 → 选行 → 「查看详情」	<code>_record_detail</code>
导出 CSV	识别记录 → 「导出 CSV」	<code>_records_export</code>
登录后预加载记录	登录成功事件链	<code>_build_records_panel_state</code>

0.2 数据库与文件

表 `records`（见 `garbage.sql`）主要字段：

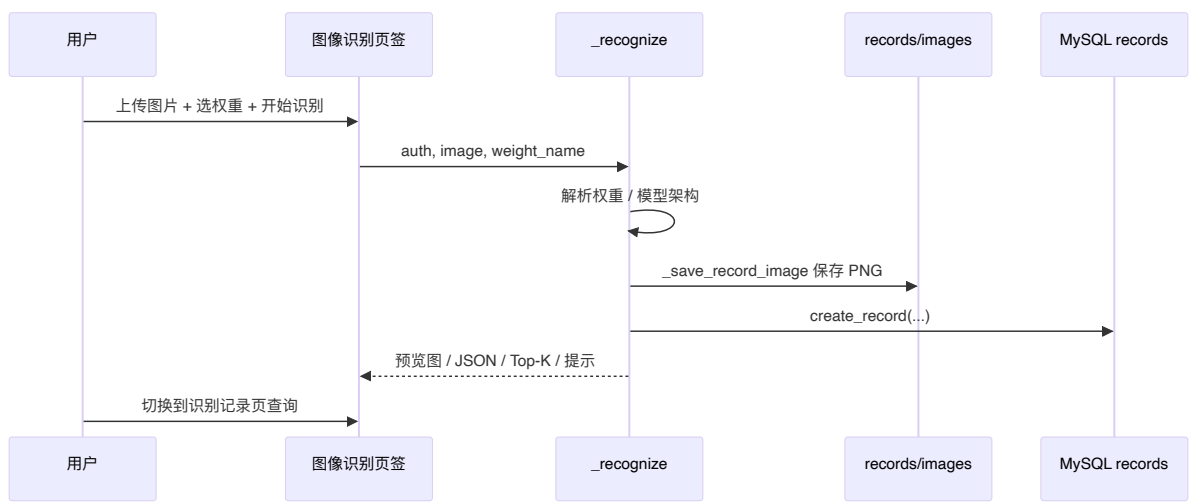
字段	含义
<code>thumb</code>	识别时保存的图片相对路径
<code>label</code>	预测类别名称
<code>confidence</code>	置信度
<code>elapsed</code>	推理耗时（秒）
<code>model</code>	模型架构（如 resnet50）
<code>weight</code>	权重文件名
<code>user</code>	登录用户名
<code>created_at</code>	识别时间

磁盘目录：

项目根目录 /	
└─ records /	
└─ images /	← 每次识别保存的 PNG 快照

类别标签：训练数据标签见 `data/label_list.txt`（如「可回收物/塑料瓶」）。当前教学版 `_recognize` 返回示例结果；选做任务可加载 `.pth` 做真实 softmax 推理（见附录 C）。

0.3 数据流（一次完整识别）



0.4 环境自检（开始编码前必做）

说明：推理页依赖模型管理的权重列表；若 `models` 表为空，下拉框无选项，但仍可用占位逻辑测试入库。

操作：

```
python -c "from gradio_app import _refresh_weights, _get_current_weight;
print('weights:', _refresh_weights()); print('current:',
_get_current_weight())"
```

确认 `records` 表与历史数据（可选）：

```
USE garbage;
SHOW TABLES LIKE 'records';
SELECT COUNT(1) FROM records;
```

确认图片目录可写：

```
python -c "import os; from gradio_app import _record_image_dir;
os.makedirs(_record_image_dir(), exist_ok=True); print(_record_image_dir())"
```

检查项	通过标准
权重列表	至少 0 个（空列表也可做占位测试）；有模型更佳
records 表	查询不报错
图片目录	路径打印正常，无权限错误

第 1 章 推理与记录后端

本章实现识别回调与记录相关数据库操作，暂不修改界面。

步骤 1.1 识别图片保存与表格工具

本节目标

把用户上传的 PIL 图片保存到项目目录，并将路径以相对路径写入数据库；同时提供把记录列表转为 Gradio 表格行的工具函数。

实现说明

- Gradio `Image(type="pil")` 传入的是 Pillow `Image` 对象，可直接 `.save()`。
- 文件名带微秒时间戳，避免连续识别互相覆盖。
- `_records_to_rows` 只投影表格需要的列，与「识别记录」页 `Dataframe` 表头一致。

操作

在 `gradio_app.py` 中追加（若登录章/模型章已有 `_items_to_rows`、`_to_relative_project_path`，勿重复定义）：

```
def _record_image_dir() -> str:
    return os.path.join(_project_root(), "records", "images")

def _save_record_image(image) -> str:
    if image is None:
        return ""
    os.makedirs(_record_image_dir(), exist_ok=True)
    filename = f"record_{datetime.now().strftime('%Y%m%d_%H%M%S_%f')}.png"
    abs_path = os.path.join(_record_image_dir(), filename)
    image.save(abs_path)
    return _to_relative_project_path(abs_path)

def _records_to_rows(items: list[dict[str, Any]]) -> list[list[Any]]:
    return _items_to_rows(items, ["id", "label", "confidence", "model",
                                   "weight", "user", "created_at"])
```

文件头部需已有：

```
import csv
import tempfile
from datetime import datetime
```

代码解读

函数	作用
<code>_save_record_image</code>	返回相对路径字符串，供 <code>records.thumb</code>
<code>_records_to_rows</code>	供 <code>_records_query</code> 输出给 <code>Dataframe</code>

验证

```
python -c "from PIL import Image; from gradio_app import _save_record_image; p=_save_record_image(Image.new('RGB',(64,64))); print('saved:', p)"
```

预期：打印 `records\images\record_....png` 形式路径。

步骤 1.2 实现 `_recognize` 识别回调

本节目标

实现 Gradio 识别按钮的核心回调：校验输入、关联权重与模型架构、生成识别结果、写入数据库。

实现说明

当前教学版逻辑（与仓库一致）：

- 1. 未上传图片 → 直接提示，不写库。
- 2. 权重名优先用下拉框选中值，为空则用 `_get_current_weight()`。
- 3. 通过 `db.get_model_arch(weight_name)` 查架构名写入 `model` 字段。
- 4. 识别结果为**固定示例**（label/confidence/topk），便于先打通 UI 与记录链。
- 5. 调用 `_save_record_image` + `create_record`，把本次识别记入 `records` 表。

返回值四元组（对应四个输出组件）：

返回值	组件
<code>image</code>	预览图 <code>rec_preview</code>
<code>result dict</code>	JSON <code>rec_result</code>
提示字符串	<code>rec_msg</code>
topk 二维列表	<code>rec_topk</code> <code>Dataframe</code>

操作

继续追加：

```
def _recognize(auth, image, weight_name):
```

```

if image is None:
    return None, None, "请先上传图片", []

weight_name = (weight_name or _get_current_weight() or "").strip()
model_name = ""
if weight_name:
    try:
        model_name = _get_db().get_model_arch(weight_name)
    except Exception:
        model_name = ""
model_name = model_name or "unknown"

result = {
    "label": "可回收物",
    "confidence": 0.95,
    "weight": weight_name,
    "model": model_name,
    "message": "这里是占位识别结果，等待学生接入真实模型。",
}

topk = [
    ["可回收物", 0.95],
    ["厨余垃圾", 0.03],
    ["其他垃圾", 0.02],
]

try:
    _get_db().create_record(
        thumb=_save_record_image(image),
        label=result["label"],
        confidence=float(result["confidence"]),
        elapsed=0.0,
        model=model_name,
        weight=weight_name,
        user=str((auth or {}).get("username") or ""),
        created_at=datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    )
except Exception as exc:
    result["message"] = f'{result["message"]} 记录保存失败: {exc}'
return image, result, "识别完成 (示例数据)", topk

```

代码解读

- `auth` 来自 `gr.State`，识别记录里的 `user` 字段取 `username`。
- `elapsed=0.0`：占位阶段未计时；接入真实模型后可用 `time.perf_counter()` 填入。
- 即使识别是示例数据，入库逻辑是真实的，「识别记录」页可立即验证。

验证

```

python -c "from PIL import Image; from gradio_app import _recognize; auth={
'username':'admin'}; print(_recognize(auth, Image.new('RGB', (32,32)),
'resnet50_20260422_161046.pth')[2])"

```

(若库中无该权重名, `model` 可能为 `unknown`, 属正常。)

步骤 1.3 记录查询、详情与导出

本节目标

为「识别记录」页签提供分页查询、按 ID 查看详情 (含图片回显)、导出 CSV 三个回调。

实现说明

- `_records_query`: 调用 `db.list_records(search, page, per_page=10)`, 返回表格行 + 分页提示文案。
- `_record_detail`: `thumb` 存的是相对路径, 展示前用 `_to_absolute_project_path` 转绝对路径, 并检查文件是否存在。
- `_records_export`: 导出到系统临时目录, 文件名带时间戳, Excel 可直接打开 (`utf-8-sig` BOM)。

操作

继续追加:

```
def _records_query(auth, q, page):
    try:
        query = (q or "").strip()
        page_num = max(int(page or 1), 1)
        rows, total_pages, total = _get_db().list_records(query, page_num,
per_page=10)
        items = [_row_to_dict(row) for row in rows]
        page_num = min(page_num, total_pages)
        return _records_to_rows(items), f"第 {page_num} 页 / 共 {total_pages}
页, 共 {total} 条记录"
    except Exception as exc:
        return [], f"加载记录失败: {exc}"

def _record_detail(auth, record_id):
    try:
        rid = int(record_id or 0)
    except (TypeError, ValueError):
        return None, None, "请输入有效的记录ID"
    if rid <= 0:
        return None, None, "请输入有效的记录ID"
    try:
        row = _get_db().get_record_by_id(rid)
        item = _row_to_dict(row)
        if not item:
            return None, None, f"未找到记录: {rid}"
        thumb_path = _to_absolute_project_path(str(item.get("thumb") or ""))
```

```

        image_value = thumb_path if thumb_path and os.path.exists(thumb_path)
    else None
    detail = {
        "id": item.get("id"),
        "label": item.get("label"),
        "confidence": item.get("confidence"),
        "elapsed": item.get("elapsed"),
        "model": item.get("model"),
        "weight": item.get("weight"),
        "user": item.get("user"),
        "thumb": item.get("thumb"),
        "created_at": item.get("created_at"),
    }
    return image_value, detail, "详情加载完成"
except Exception as exc:
    return None, None, f"加载详情失败: {exc}"

def _records_export(auth, q):
    try:
        query = (q or "").strip()
        items = [_row_to_dict(row) for row in
_get_db().export_records(query)]
        export_dir = os.path.join(tempfile.gettempdir(),
"garbage_classifier_exports")
        os.makedirs(export_dir, exist_ok=True)
        export_path = os.path.join(export_dir,
f"records_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv")
        with open(export_path, "w", newline="", encoding="utf-8-sig") as f:
            writer = csv.writer(f)
            writer.writerow(["id", "thumb", "label", "confidence", "elapsed",
"model", "weight", "user", "created_at"])
            for item in items:
                writer.writerow([
                    item.get("id"), item.get("thumb"), item.get("label"),
                    item.get("confidence"), item.get("elapsed"),
                    item.get("model"),
                    item.get("weight"), item.get("user"),
                    item.get("created_at"),
                ])
            return export_path, f"导出完成, 共 {len(items)} 条记录"
    except Exception as exc:
        return None, f"导出失败: {exc}"

```

验证

```

python -c "from gradio_app import _records_query;
rows,info=_records_query(None,'',1); print(info, 'rows=', len(rows))"

```

步骤 1.4 记录面板状态整理

`_build_records_panel_state`

本节目标

登录成功或需要刷新记录页时，一次性更新表格、分页信息，并清空详情区，避免显示上一条记录的残留。

实现说明

返回 6 个值，对应 `build_main_panel` 中 `records_outputs` 的 6 个组件：表格、信息、记录 ID、详情图、详情 JSON、详情提示。

操作

继续追加：

```
def _build_records_panel_state(auth, query: str = "", page: int = 1):
    rows, info = _records_query(auth, query, page)
    return rows, info, 0, None, None, ""
```

验证

完成第 2 章 UI 与 `build_app` 绑定后，登录即应触发此函数（IN7）。

第 2 章 界面与事件绑定

在 `build_main_panel` 的 `with gr.Tabs():` 内增加「图像识别」「识别记录」两个页签（若已有模型管理页签，在其前或后追加即可）。

步骤 2.1 「图像识别」页签

本节目标

提供上传图片、选择权重、触发识别、展示 JSON 与 Top-K 的完整界面。

实现说明

- `rec_image` 使用 `type="pil"`，与 `_save_record_image` 一致。
- `rec_weight` 初始化时读 `callbacks["refresh_weights"]()` 与 `get_current_weight()`，与模型管理共用数据源。
- `_weights_update` 在用户点击「刷新权重列表」时更新下拉选项，并尽量保持当前选中项。

操作

```

with gr.Tab("图像识别"):
    rec_image = gr.Image(label="上传图片", type="pil")
    rec_weight = gr.Dropdown(
        label="权重文件",
        choices=callbacks["refresh_weights"](),
        value=callbacks["get_current_weight"]() or None,
    )
    rec_refresh = gr.Button("刷新权重列表")
    rec_btn = gr.Button("开始识别")
    rec_msg = gr.Textbox(label="提示", interactive=False)
    rec_preview = gr.Image(label="预览", interactive=False)
    rec_result = gr.JSON(label="识别结果")
    rec_topk = gr.Dataframe(label="Top-K", headers=["label", "score"],
        interactive=False)

    def _weights_update():
        choices = callbacks["refresh_weights"]()
        cur = callbacks["get_current_weight"]() or (choices[0] if choices
    else None)
        return gr.update(choices=choices, value=cur)

    rec_refresh.click(_weights_update, inputs=None, outputs=[rec_weight])
    rec_btn.click(
        callbacks["recognize"],
        inputs=[auth_state, rec_image, rec_weight],
        outputs=[rec_preview, rec_result, rec_msg, rec_topk],
    )

```

验证

启动应用，登录后进入「图像识别」页签，控件齐全（IN1 目视）。

步骤 2.2 「识别记录」页签

本节目标

实现记录分页列表、点击行选中 ID、查看详情、导出 CSV。

实现说明

- `rec_table.select`：Gradio 表格选中事件，用 `_pick_record_id` 把该行第一列 `id` 写入 `rec_id` 数字框，减少手动输入。
- 查询与导出共用搜索框 `rec_q` 作为关键字。
- 详情区三个组件：图片、JSON、提示文本。

操作

```

with gr.Tab("识别记录"):

```

```

rec_q = gr.Textbox(label="搜索")
rec_page = gr.Number(label="页码", value=1, precision=0)
rec_query_btn = gr.Button("查询")
rec_info = gr.Textbox(label="信息", interactive=False)
rec_table = gr.Dataframe(
    label="记录列表",
    headers=["id", "label", "confidence", "model", "weight", "user",
"created_at"],
    interactive=False,
)
rec_id = gr.Number(label="记录ID", precision=0)
rec_detail_btn = gr.Button("查看详情")
rec_detail_img = gr.Image(label="图片", interactive=False)
rec_detail_json = gr.JSON(label="详情")
rec_detail_msg = gr.Textbox(label="详情提示", interactive=False)
export_btn = gr.Button("导出CSV (管理员)")
export_file = gr.File(label="导出文件")
export_msg = gr.Textbox(label="导出提示", interactive=False)

def _pick_record_id(table_value, evt: gr.SelectData):
    if not evt or not getattr(evt, "index", None):
        return 0
    row_index = evt.index[0] if isinstance(evt.index, (list, tuple)) else
evt.index
    if isinstance(table_value, list) and 0 <= row_index <
len(table_value):
        row = table_value[row_index]
        if isinstance(row, (list, tuple)) and row:
            try:
                return int(row[0] or 0)
            except (TypeError, ValueError):
                return 0
    return 0

rec_query_btn.click(
    callbacks["records_query"],
    inputs=[auth_state, rec_q, rec_page],
    outputs=[rec_table, rec_info],
)
rec_table.select(_pick_record_id, inputs=[rec_table], outputs=[rec_id])
rec_detail_btn.click(
    callbacks["record_detail"],
    inputs=[auth_state, rec_id],
    outputs=[rec_detail_img, rec_detail_json, rec_detail_msg],
)
export_btn.click(
    callbacks["records_export"],
    inputs=[auth_state, rec_q],
    outputs=[export_file, export_msg],
)

```

`build_main_panel` 返回值补充:

```
"records_outputs": [rec_table, rec_info, rec_id, rec_detail_img,
rec_detail_json, rec_detail_msg],
```

验证

登录后切换到「识别记录」，点击「查询」能看到 `garbage.sql` 中的历史记录（IN7）。

步骤 2.3 更新 `build_app`：注册 callbacks 与登录后加载

本节目标

把推理相关函数注册到 `callbacks` 字典；登录成功后自动加载记录列表。

实现说明

在 `build_main_panel` 的 `callbacks` 中增加：

```
"recognize": _recognize,
"records_query": _records_query,
"record_detail": _record_detail,
"records_export": _records_export,
```

登录按钮事件链末尾追加（与模型页 `_load_models_ui` 并列）：

```
).then(
    _build_records_panel_state,
    inputs=[auth_state],
    outputs=main_ui["records_outputs"],
)
```

这样用户登录后，「识别记录」页无需手动点查询即可看到最新列表（含 `garbage.sql` 样例数据）。

验证

`admin` 登录 → 打开「识别记录」→ 表格已有数据（IN7）。

第 3 章 功能测试

3.1 运行

```
python gradio_app.py
```

使用 `admin / 123` 登录。

3.2 测试用例

编号	操作	预期结果
IN1	进入「图像识别」	可见上传区、权重下拉、开始识别按钮
IN2	不上传图片点「开始识别」	提示「请先上传图片」，无新记录
IN3	上传图片并识别	JSON 含 label/confidence；Top-K 有三行；提示「识别完成」
IN4	识别后到「识别记录」查询	表格最上方出现刚产生的记录， <code>user</code> 为 admin
IN5	点击表格某行 → 「查看详情」	详情 JSON 完整；图片区显示快照（若文件存在）
IN6	搜索关键字（如「可回收」）后查询	列表过滤；分页信息更新
IN7	重新登录	无需点查询，记录表自动有数据
IN8	点击「导出 CSV」	下载 CSV，行数与当前筛选一致

MySQL 验证：

```
SELECT id, label, confidence, user, thumb, created_at FROM records ORDER BY id DESC LIMIT 5;
```

磁盘验证：

```
records/images/record_*.png
```